UNIVERSIDAD AUTÓNOMA DE OCCIDENTE
Facultad de Ingeniería
Proyecto Informático 1 · 2026-2

INFORME DE PRÁCTICAS DE INGENIERÍA: CONTROL DE VERSIONES, PULL REQUESTS Y REVISIÓN DE CÓDIGO

Proyecto CanchaYa · Sprint Review #3 y #4

Equipo de desarrollo

Samuel Ríos Calderón (2230205)	GitHub: Samuel Rios
Juan Esteban Panesso Bernal (2215930)	GitHub: JuanAZ-uao
Juan Sebastián Castillo Acevedo (2231921)	GitHub: Juansec01
Byron Stiven Zapata Zapata (2210718)	GitHub: ByronZapata2003

Docente: Jorge González Rivera

Repositorio: https://github.com/JuanAZ-uao/proyecto_informaticocanchaya

Cali · 14 de octubre de 2026

CONTENIDO

1. Tablero de la Sprint Planning y sprint backlog (Sprint #3 y #4)
2. Repositorio, README y .gitignore
3. Tabla resumen de Pull Requests
4. Evidencias del historial de Git del ciclo (Sprint #3)
5. Preguntas guía sobre las prácticas de ingeniería
6. Calidad del incremento: pruebas, SonarQube y documentación de la API
7. Declaración de uso de inteligencia artificial
8. Estado de cumplimiento de la guía y mejoras

1. TABLERO DE LA SPRINT PLANNING Y SPRINT BACKLOG

La Sprint Planning #3 definió el alcance del motor de reservas (validación de disponibilidad, bloqueo de duplicados, flujo rápido de reserva y temporizador por socket) más el plan de pruebas de QA. Las figuras 1 y 2 muestran el tablero de Jira con el listado de historias de usuario (HU) del sprint backlog actual: el Sprint 3 (30 de septiembre al 13 de octubre) y el Sprint 4 (14 al 27 de octubre).

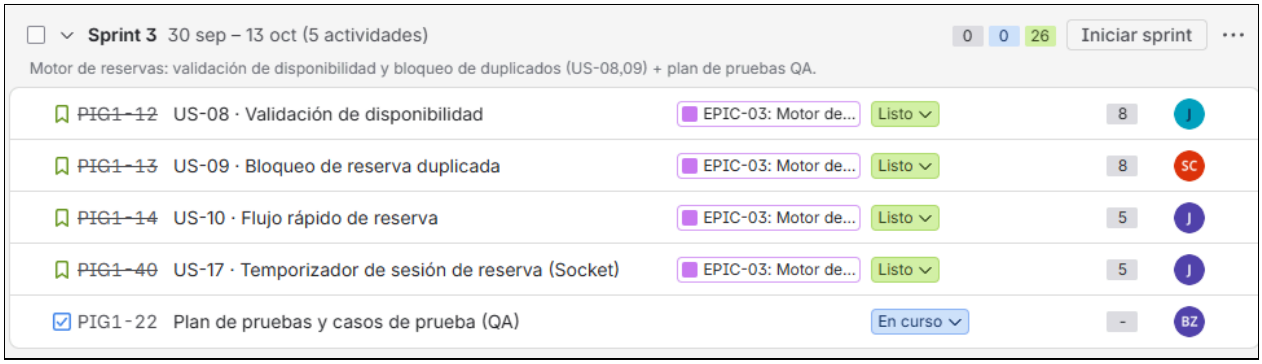


Figura 1. Tablero de Jira: Sprint Backlog del Sprint 3 (30 sep – 13 oct), 5 actividades.



Figura 2. Tablero de Jira: Sprint Backlog del Sprint 4 (14 oct – 27 oct), 8 actividades.

Tabla 1. Historias de usuario de los sprints 3 y 4 según el tablero

Sprint	Clave Jira	Historia de usuario	Puntos
3	PIG1-12	US-08 · Validación de disponibilidad	8
3	PIG1-13	US-09 · Bloqueo de reserva duplicada	8
3	PIG1-14	US-10 · Flujo rápido de reserva	5
3	PIG1-40	US-17 · Temporizador de sesión de reserva (Socket)	5

Sprint	Clave Jira	Historia de usuario	Puntos
3	PIG1-22	Plan de pruebas y casos de prueba (QA)	—
4	PIG1-15	US-11 · Confirmación de reserva	3
4	PIG1-16	US-12 · Reservas múltiples	3
4	PIG1-17	US-13 · Historial de reservas	3
4	PIG1-41	US-18 · Retención de franja y disponibilidad en tiempo real (Socket)	5
4	PIG1-23	Documentación técnica de API (OpenAPI/Swagger)	—
4	PIG1-42	US-19 · Contenedorización con Docker	5
4	PIG1-43	US-20 · Pipeline CI/CD con Docker	5
4	PIG1-44	US-21 · Integración de SonarQube y continuación de pruebas QA	5

2. REPOSITORIO, README Y .GITIGNORE

Repositorio público del proyecto: https://github.com/JuanAZ-uao/proyecto_informaticocanchaya. Contiene dos proyectos npm independientes (backend y frontend), la configuración de Docker Compose, el pipeline de GitHub Actions, las pruebas de QA y la documentación.

2.1 README.md

El README (README.md) incluye lo que pide la guía:

- **Descripción del proyecto:** aplicación web para consultar canchas sintéticas, ver su disponibilidad y reservar franjas de una hora, sin reservas duplicadas y con disponibilidad en tiempo real.
- **Stack tecnológico:** React 18 con Vite, Node.js 22 con Express, PostgreSQL, Socket.IO, Docker Compose, GitHub Actions, Jest, Playwright, SonarQube y OpenAPI/Swagger.
- **Estrategia de ramas con diagrama:** *main* (versión validada), *sprint_N* (integración del sprint), *feature/us-n-descripcion* (una rama por historia), *fix/* y *qa/*. La figura 3 resume el diagrama.
- **Convención de commits:** Conventional Commits con los tipos feat, fix, docs, test, refactor, style, build, ci, chore y perf, con el formato tipo(alcance): descripción corta en imperativo.
- **Proceso de Pull Request, método de fusión y resolución de conflictos.**
- Además, la evolución del proyecto por sprint con capturas del tablero de Jira, la documentación de la API y el resumen de las pruebas de QA.

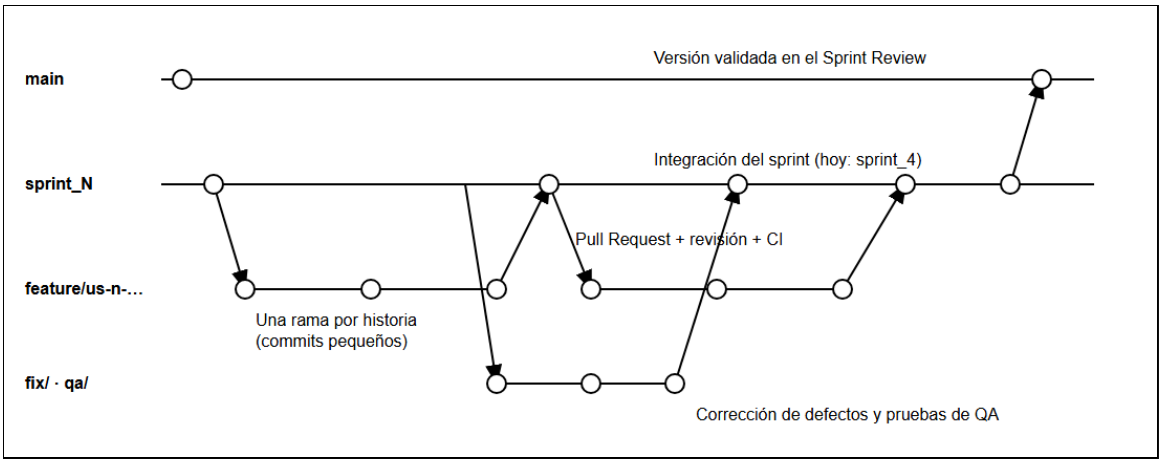


Figura 3. Estrategia de ramas del equipo (diagrama del README).

2.2 Archivo .gitignore

El .gitignore de la raíz, junto con los de backend y frontend, excluye lo que no debe versionarse en un proyecto Node.js con Docker:

- **Dependencias y artefactos:** node_modules/, dist/, build/ y coverage/.
- **Variables de entorno:** .env y .env.* (con la excepción !.env.example). El archivo .env real contiene credenciales de la base de datos y nunca se sube; solo se versionan los .env.example sin secretos.
- **Registros:** npm-debug.log* y *.log.
- **Sistema operativo y editores:** .DS_Store, Thumbs.db y .idea/.
- **SonarQube:** .scannerwork/, report-task.txt y .sonar_lock.

2.3 Ramas del repositorio y su relación con las historias de usuario

Tabla 2. Ramas remotas y la historia de usuario que desarrollan

Rama	Historia de usuario	Integrante
feature/us-08-validacion-disponibilidad	US-08	Juan Sebastián Castillo
US-09/Bloqueo_reserva_duplicada	US-09	Samuel Ríos
US-07/Selección_de_fecha_hora	US-07 (Sprint 2)	Samuel Ríos
H.U-06-Filtro_de_canchas	US-06 (Sprint 2)	Byron Zapata

Rama	Historia de usuario	Integrante
feature/ci-cd-pipeline	CI/CD inicial (Sprint 2)	Juan Sebastián Castillo
US--11,18,19,20	US-11, US-18, US-19, US-20	Juan Esteban Panesso
feature/us-13-historial-reservas	US-13	Juan Sebastián Castillo
US-12/Reservas_multiples	US-12	Samuel Ríos
US-21_SonarQube_y_continuacion_de_pruebas_QA	US-21	Byron Zapata
qa/sprint-3-testing	PIG1-22 (plan de pruebas del Sprint 3)	Byron Zapata
sprint_1 · sprint_2 · sprint_3 · sprint_4	Ramas de integración de cada sprint	Equipo

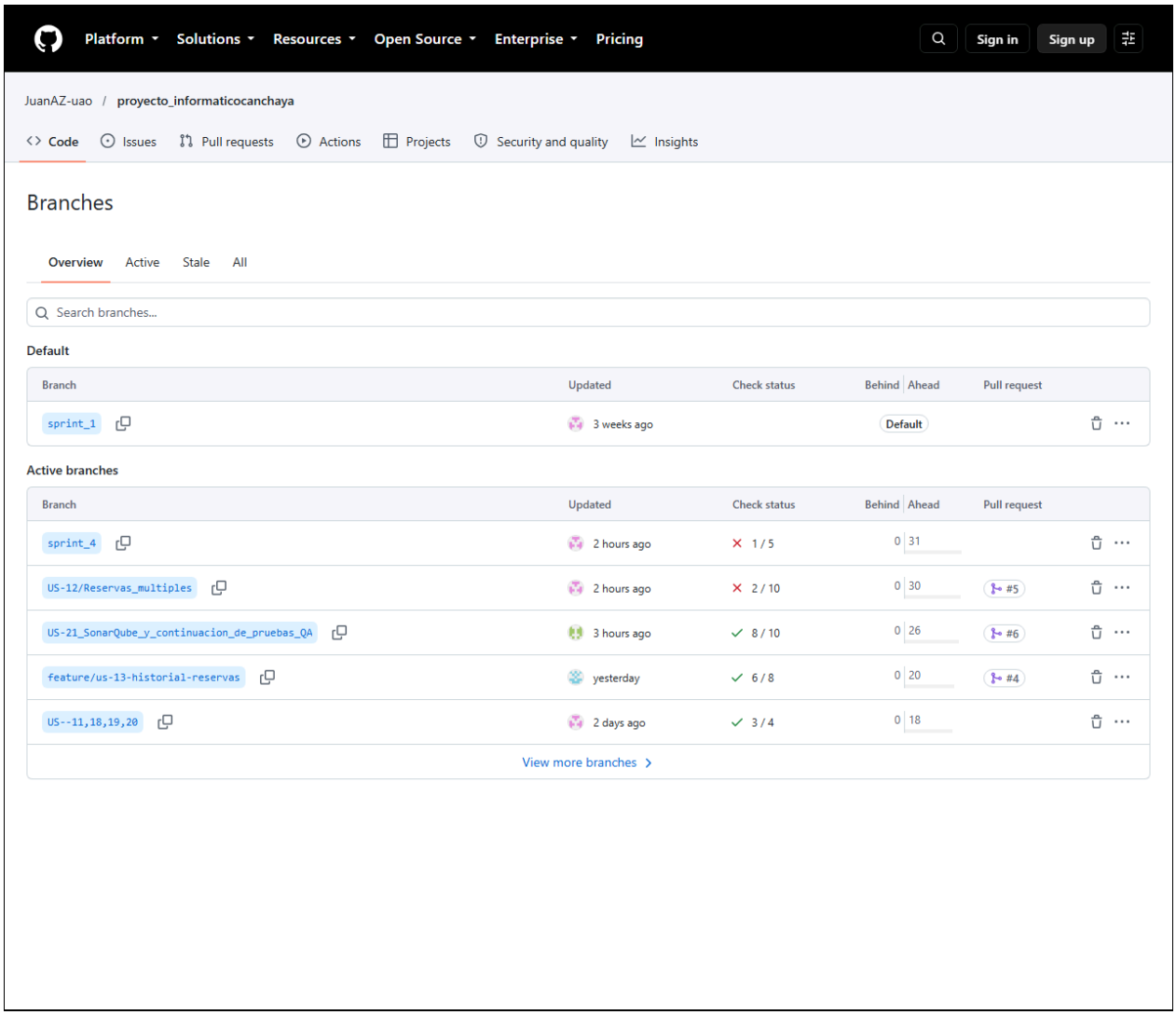


Figura 4. Página de ramas del repositorio en GitHub.

3. TABLA RESUMEN DE PULL REQUESTS

La tabla 3 lista todos los Pull Requests del repositorio al 14 de octubre de 2026, con la URL pública de cada uno. La columna *Revisor* refleja lo que GitHub tiene registrado: ningún PR tiene aún revisiones ni comentarios de otra persona, por lo que se indica quién fusionó cada uno y el resultado de los checks automáticos.

Tabla 3. Pull Requests del repositorio

#PR	HU relacionada	Autor	Revisor	Método de merge	Enlace
#1	Tarea técnica de CI/CD (antecedente de US-20)	Juan Sebastián Castillo (Juansec01)	No registrado en GitHub	Cerrado sin fusionar en GitHub; el contenido se integró con git merge local en sprint_2	https://github.com/JuanAZ-uao/proyecto_informaticocanchaya/pull/1
#2	US-08	Juan Sebastián Castillo (Juansec01)	Fusionó JuanAZ-uao; sin comentarios de revisión	Integración local (fast-forward) y push; GitHub lo marcó Merged	https://github.com/JuanAZ-uao/proyecto_informaticocanchaya/pull/2
#3	US-11, US-18, US-19, US-20	Juan Esteban Panesso (JuanAZ-uao)	Sin revisión registrada; 8 de 8 checks de CI en verde	Create a merge commit	https://github.com/JuanAZ-uao/proyecto_informaticocanchaya/pull/3
#4	US-13	Juan Sebastián Castillo (Juansec01)	Fusionó JuanAZ-uao; sin comentarios de revisión	Create a merge commit	https://github.com/JuanAZ-uao/proyecto_informaticocanchaya/pull/4

#PR	HU relacionada	Autor	Revisor	Método de merge	Enlace
#5	US-12	Samuel Ríos (commit de la HU) · PR abierto por JuanAZ-uao	Fusionó JuanAZ-uao; sin comentarios de revisión	Create a merge commit (tras resolver un conflicto)	https://github.com/JuanAZ-uao/proyecto_informaticocanchaya/pull/5
#6	US-21	Byron Zapata (commits) · PR abierto por JuanAZ-uao	Fusionó JuanAZ-uao; sin comentarios de revisión	Create a merge commit	https://github.com/JuanAZ-uao/proyecto_informaticocanchaya/pull/6

Nota: el código de las historias integradas por los PR #5 y #6 lo escribieron Samuel Ríos (US-12) y Byron Zapata (US-21); el PR lo abrió la cuenta JuanAZ-uao porque era quien integraba la rama del sprint.

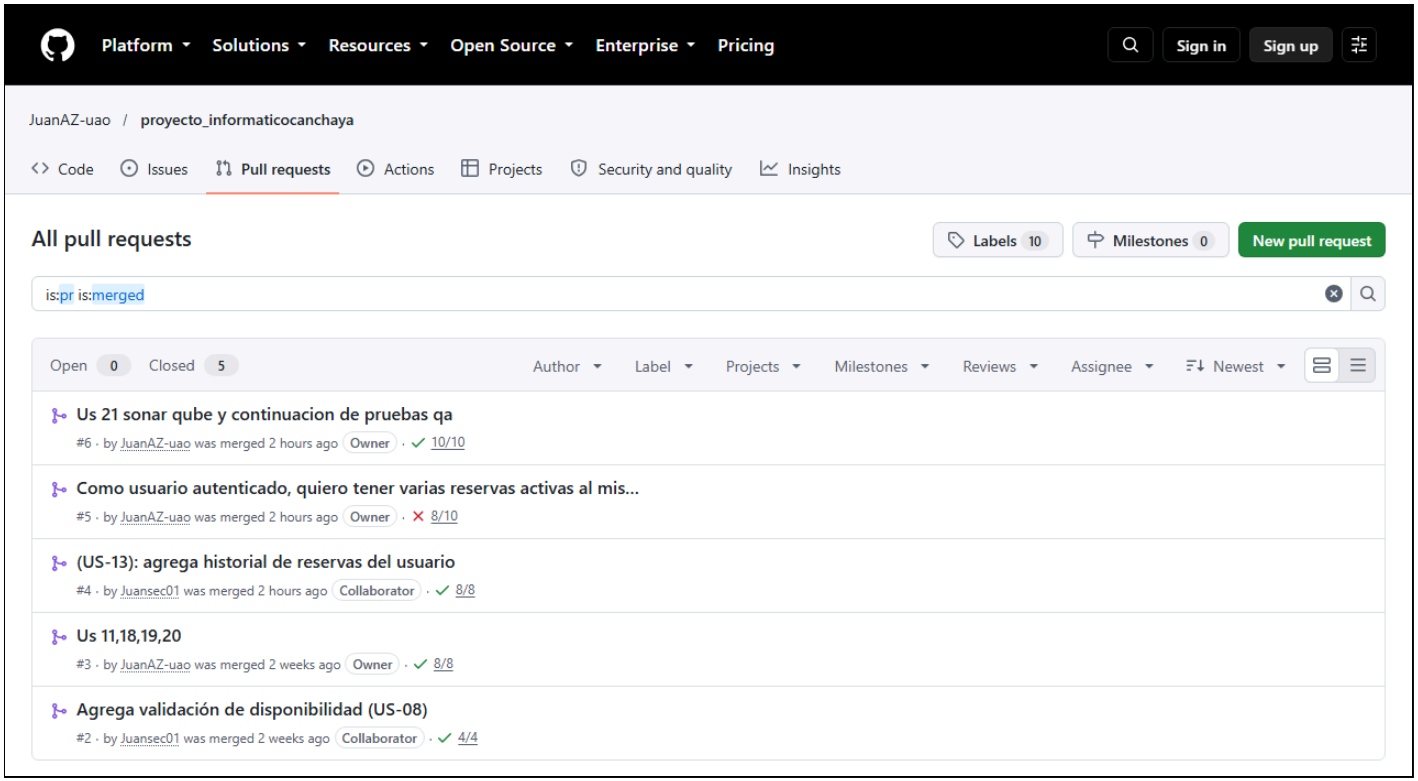


Figura 5. Pestaña Pull requests de GitHub filtrada con is:pr is:merged: PR fusionados.

4. EVIDENCIAS DEL HISTORIAL DE GIT DEL CICLO (SPRINT #3)

El repositorio tiene 41 commits en todas sus ramas. El comando git shortlog -sn --all muestra el aporte de cada integrante, lo que respalda que los cuatro trabajaron con commits propios:

Tabla 4. Commits por integrante (git shortlog -sn --all)

Cuenta de Git	Integrante	Commits
JuanAZ-uao	Juan Esteban Panesso Bernal	26
ByronZapata2003	Byron Stiven Zapata Zapata	9
Juansec01	Juan Sebastián Castillo Acevedo	3
Samuel Rios	Samuel Ríos Calderón	3

4.1 Commits trazables a la tarea de su historia de usuario

La tabla siguiente recoge los commits del ciclo (desde el 29 de septiembre) y la historia de usuario a la que corresponde cada uno. La trazabilidad HU → rama → commit existe en los nombres de las ramas y en los PR. Como autocrítica, varios commits agrupan más de una historia (por ejemplo, 3fdf148 con US-08, US-09, US-10 y US-17) y seis commits de la rama de US-21 conservan como mensaje la salida de git status. La convención de commits atómicos con tipo y alcance quedó definida en el README y se aplica desde este punto.

Tabla 5. Commits del ciclo y su historia de usuario

Commit	Autor	Fecha	Mensaje	HU / tarea
3fdf148	J. E. Panesso	2026-09-29	Implementacion de las historias de usuario 08,09,10 y 17 que es el ini…	US-08, US-09, US-10, US-17
ec48c97	B. Zapata	2026-09-29	new file: docs/testing/SPRINT_3_TEST_PLAN.md new file: docs/testing/SP…	PIG1-22 (plan de pruebas)

Commit	Autor	Fecha	Mensaje	HU / tarea
5b9d8d3	J. E. Panesso	2026-09-29	Implementacion de las historias de usuario 08,09,10 y 17 que es el ini…	US-08, US-09, US-10, US-17
8a4716a	J. E. Panesso	2026-09-29	merge de la US-08 y US-09, mas prueba de que funcione CI/CD	US-08, US-09
4d07a4d	J. E. Panesso	2026-09-29	ci: ejecutar el pipeline tambien en cada push a sprint_3	US-20 (CI/CD)
543243a	J. E. Panesso	2026-09-30	Merge pull request #3 from JuanAZ-uao/US--11,18,19,20	US-11, US-18, US-19, US-20 (merge del PR #3)
ee309ca	J. E. Panesso	2026-09-30	Update ci-cd.yml	US-20 (CI/CD)
548e6ad	J. E. Panesso	2026-09-30	Historias de usuariop 11, 18,19,20 mas cambio en el readme, para mayor…	US-11, US-18, US-19, US-20
1780736	J. E. Panesso	2026-10-11	correcion de backend y frontend	Correcciones de QA (DEF-01 a DEF-10)
351c9d0	J. S. Castillo	2026-10-12	(US-13): agrega historial de reservas del usuario	US-13
eb180bc	S. Ríos	2026-10-12	Como usuario autenticado, quiero tener varias reservas activas al mism…	US-12
52c31a6	J. E. Panesso	2026-10-13	Merge pull request #5 from JuanAZ-uao/US-12/Reservas_multiples	US-12 (merge del PR #5)

Commit	Autor	Fecha	Mensaje	HU / tarea
be08e91	J. E. Panesso	2026-10-13	Merge branch 'sprint_4' into US-12/Reservas_multiples	US-12 (resolución del conflicto)
fe996dc	J. E. Panesso	2026-10-13	Merge pull request #6 from JuanAZ-uao/US-21_SonarQube_y_continuacion_d…	US-21 (merge del PR #6)
7910419	J. E. Panesso	2026-10-13	Merge pull request #4 from JuanAZ-uao/feature/us-13-historial-reservas	US-13 (merge del PR #4)
7e8202f	B. Zapata	2026-10-13	modified: backend/package.json	US-21
b51b71c	B. Zapata	2026-10-13	renamed: scripts/pruebaCargaSockets.js -> backend/scripts/pruebaCargaS…	US-21
07d8ce2	B. Zapata	2026-10-13	modified: .gitignore modified: frontend/.gitignore	US-21
20baa05	B. Zapata	2026-10-13	renamed: backend/scripts/ci/prefijar-lcov.mjs -> scripts/ci/prefijar-l…	US-21
6576922	B. Zapata	2026-10-13	modified: backend/.gitignore	US-21
618e3ac	B. Zapata	2026-10-13	modified: .github/workflows/ci-cd.yml new file: backend/jest.config.js…	US-21

```
$ git log --oneline --graph --all
* d1ed711 Merge origin/sprint_4 (US-12 reservas múltiples) en sprint_4
|\
| * 52c31a6 Merge pull request #5 from JuanAZ-uao/US-12/Reservas_multiples
| |\
| | * be08e91 Merge branch 'sprint_4' into US-12/Reservas_multiples
| | |\
| | |/
| |/|
* | | 39f2c0d cambios con las otras h.u
|\ \ \
| |/ /
|/| /
| |/
| * eb180bc Como usuario autenticado, quiero tener varias reservas activas al mismo tiempo, para organizar distintos partidos sin
* | fe996dc Merge pull request #6 from JuanAZ-uao/US-21_SonarQube_y_continuacion_de_pruebas_QA
|\ \
| * | 7e8202f modified: backend/package.json
| * | b51b71c renamed: scripts/pruebaCargaSockets.js -> backend/scripts/pruebaCargaSockets.js
| * | 07d8ce2 modified: .gitignore modified: frontend/.gitignore
| * | 20baa05 renamed: backend/scripts/ci/prefijar-lcov.mjs -> scripts/ci/prefijar-lcov.mjs renamed: backend/scripts/pruebaCargaSo
| * | 6576922 modified: backend/.gitignore
| * | 618e3ac modified: .github/workflows/ci-cd.yml new file: backend/jest.config.js new file: backend/scripts/ci/prefijar-lcov.mj
* | | 7910419 Merge pull request #4 from JuanAZ-uao/feature/us-13-historial-reservas
|\| |
| |/
|/|
| * 351c9d0 (US-13): agrega historial de reservas del usuario
|/
* 543243a Merge pull request #3 from JuanAZ-uao/US--11,18,19,20
|\
| * ee309ca Update ci-cd.yml
| | * 1780736 correcion de backend y frontend
| |/
| * 548e6ad Historias de usuariop 11, 18,19,20 mas cambio en el readme, para mayor contexto para el que mire el proyecto, termino
|/
* 3fdf148 Implementacion de las historias de usuario 08,09,10 y 17 que es el inicio del socket.io, mas actualizacion del frontend
| * ec48c97 new file: docs/testing/SPRINT_3_TEST_PLAN.md new file: docs/testing/SPRINT_3_TEST_RESULTS.md
|/
* 5b9d8d3 Implementacion de las historias de usuario 08,09,10 y 17 que es el inicio del socket.io
* 8a4716a merge de la US-08 y US-09, mas prueba de que funcione CI/CD
* 4d07a4d ci: ejecutar el pipeline tambien en cada push a sprint_3
* c2f18cb merge de la US-08 y US-09, mas prueba de que funcione CI/CD
* c68902a Merge remote-tracking branch 'origin/US-09/Bloqueo_reserva_duplicada' into sprint_3
|\
| * b071329 Como usuario autenticado, quiero que el sistema bloquee automáticamente cualquier intento de reservar la misma cancha,
* | 8f7aee5 Agrega validación de disponibilidad (US-08) y conecta CI/CD a sprint_3
|/
* cd9f942 sprint2, con el merge de todas las ramas y funcionamiento conjunto en un solo branch
* 3b932be Merge remote-tracking branch 'origin/feature/ci-cd-pipeline' into sprint_2
|\
| * 135703a Agrega flujo de ramas y pipeline inicial de GitHub Actions
```

Figura 6. Historial de Git (git log --oneline --graph --all): ramas de las HU y sus merges en la rama del sprint.

4.2 Ramas de funcionalidad vinculadas a su historia y un PR fusionado por integrante

Tabla 6. Pull Request fusionado por integrante

Integrante	Rama de la historia	PR fusionado	Historia
Juan Esteban Panesso	US--11,18,19,20	PR #3	US-11, US-18, US-19, US-20

Integrante	Rama de la historia	PR fusionado	Historia
Juan Sebastián Castillo	feature/us-08-validacion-disponibilidad · feature/us-13-historial-reservas	PR #2 · PR #4	US-08 · US-13
Samuel Ríos	US-12/Reservas_multiples	PR #5	US-12
Byron Zapata	US-21_SonarQube_y_continuacion_de_pruebas_QA	PR #6	US-21

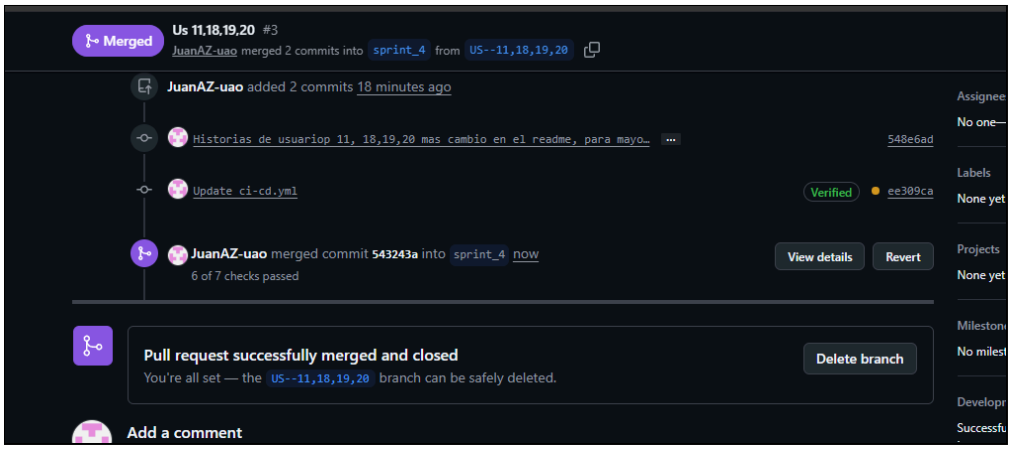


Figura 7. PR #3 en estado Merged (morado): fusionó 2 commits de la rama US--11,18,19,20 en sprint_4; 6 de 7 checks en verde.



Figura 8. Lista de Pull Requests con su estado y resultado de los checks automáticos (8 de 10 en el #5, con error; 10 de 10 en el #6).

4.3 Pull Request con comentarios o solicitudes de modificación antes de la aprobación

Estado: pendiente. A la fecha de este informe ningún PR del repositorio tiene comentarios de revisión ni solicitudes de cambios de otra persona (la API de GitHub devuelve 0 revisiones y 0 comentarios en los PR #1 a #6). Lo que sí revisó cada PR fueron los checks automáticos del pipeline (backend, frontend, SonarQube y Docker): en el PR #5 esos checks marcaron 8 de 10 y alertaron de un problema que luego se corrigió (sección 4.4). El equipo debe registrar la revisión cruzada en el siguiente PR para cumplir este criterio; el procedimiento está descrito en la sección 8.

4.4 Conflicto de fusión y su corrección

Qué pasó. Al integrar la historia US-12 (rama US-12/Reservas_multiples), GitHub mostró el aviso «This branch has conflicts that must be resolved» en el PR #5, porque la rama del sprint (sprint_4) había avanzado con otras historias (US-11, US-18, US-19, US-20, US-13 y US-21) desde que se creó la rama de la historia.

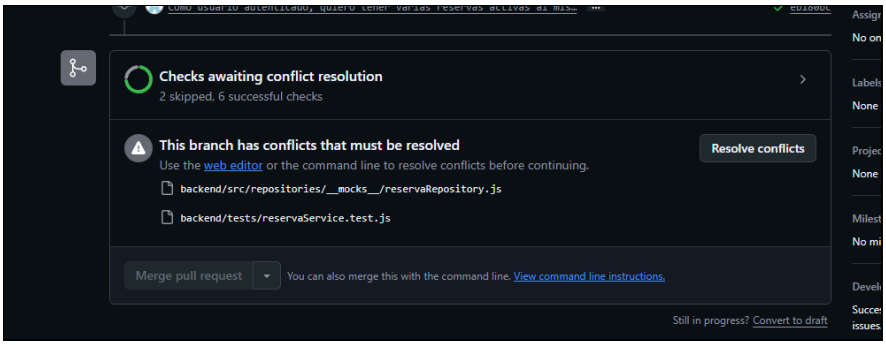


Figura 9. PR #5: GitHub bloquea el botón Merge pull request hasta resolver los conflictos en dos archivos.



Figura 10. Editor de conflictos de GitHub: marcadores del conflicto en reservaRepository.js (mock de pruebas).

Tabla 7. Archivos y líneas que entraron en conflicto

Archivo	Líneas	Causa del conflicto
backend/src/repositories/__mocks__/reservaRepository.js	1 a 14	sprint_4 tenía todo el export del mock en una sola línea; US-12 lo reescribió en varias líneas y agregó los métodos existeSolapadaDeUsuario y contarConfirmadasPorUsuario. Los dos cambios tocaron las mismas líneas.

Archivo	Líneas	Causa del conflicto
backend/tests/reservaService.test.js	Bloque de datosReserva y mocks de beforeEach	US-12 usaba una fecha calculada (fechaFutura) y mocks nuevos; sprint_4 cambió la fecha y agregó pruebas de US-18 en el mismo bloque.

Comandos usados para identificarlo y resolverlo (en el repositorio local, sobre la rama sprint_4):

- git fetch origin para traer los cambios de GitHub.
- git merge origin/sprint_4 para fusionar y provocar el conflicto.
- git status y git diff --name-only --diff-filter=U para listar los archivos en conflicto.
- git add archivo1 archivo2 (solo los archivos resueltos, sin git add .) y git commit para cerrar el merge.

Cómo decidimos qué conservar. En el mock se conservaron todos los métodos de ambas versiones (los de US-12 y los de sprint_4), porque cada historia los necesita. En el archivo de pruebas se mantuvieron las pruebas de US-18 y las nuevas de US-12 y se dejó una sola fecha futura. Al revisar el resultado se vio que la resolución hecha en la interfaz web de GitHub (opción «Accept both changes») había dejado marcadores de conflicto anidados dentro del código ya fusionado (por eso los checks del PR #5 marcaron 8 de 10): se eliminaron los marcadores y se unificaron los bloques a mano.

Cómo verificamos que todo funcionaba. Se buscaron marcadores residuales en todo el repositorio, se ejecutaron las pruebas de backend con una base de datos real (92 pruebas en verde al resolver el conflicto) y la regresión completa de QA (funcionales, E2E, no funcionales y smoke de Docker). Esa verificación mostró además que la historia US-13 había quedado incompleta tras la fusión y que las pruebas de US-12 no respetaban la validación de horario; ambos defectos (DEF-11 y DEF-13 del informe de QA) se corrigieron.

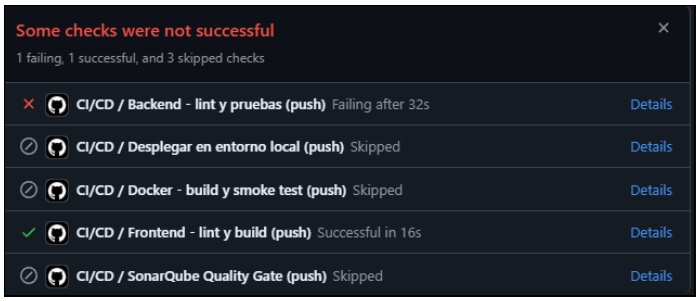


Figura 11. Checks del push: el job «Backend - lint y pruebas» falla mientras el conflicto estaba mal resuelto.

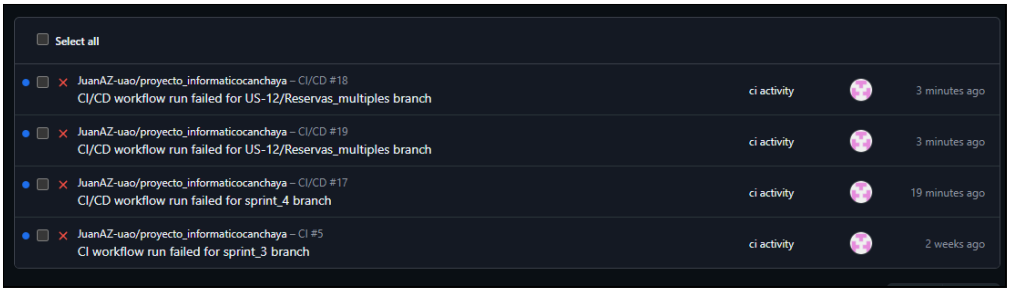


Figura 12. Notificaciones de GitHub Actions: el pipeline falló en la rama US-12/Reservas_multiples y en sprint_4.

4.5 Integración continua como control de calidad del PR

Cada PR dispara el pipeline de GitHub Actions. Las figuras siguientes muestran los checks de un PR fusionado correctamente (backend, frontend, Docker y despliegue en verde) y el detalle de cada job.

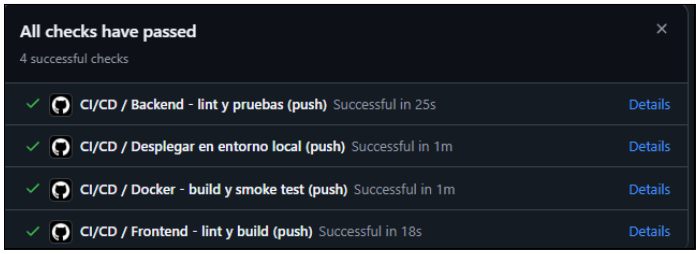


Figura 13. Los 4 checks del pipeline en verde tras la fusión en sprint_4.

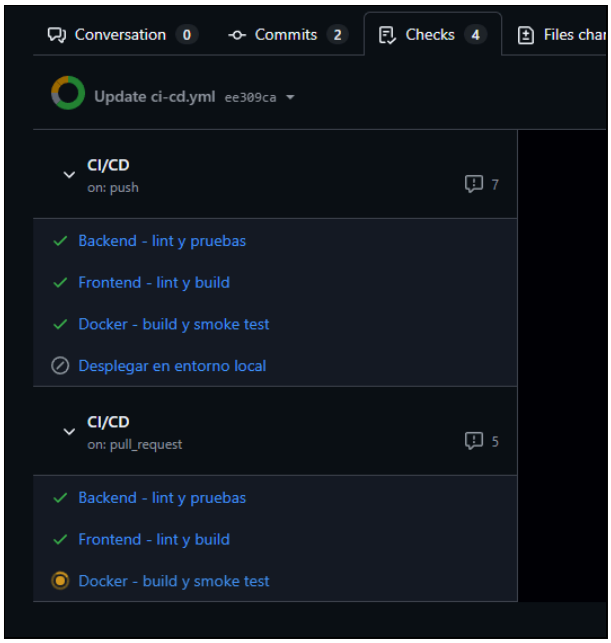


Figura 14. Checks del PR #3 en las ejecuciones push y pull_request.

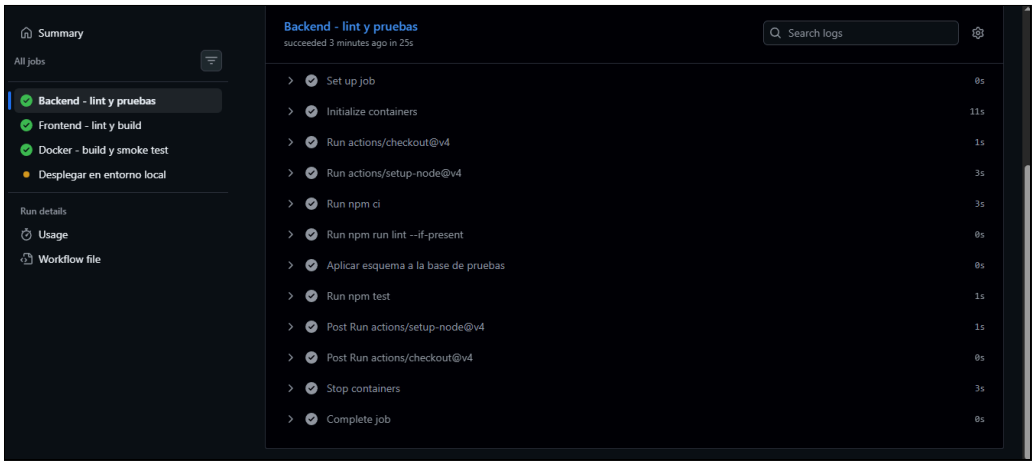


Figura 15. Detalle del job Backend - lint y pruebas.

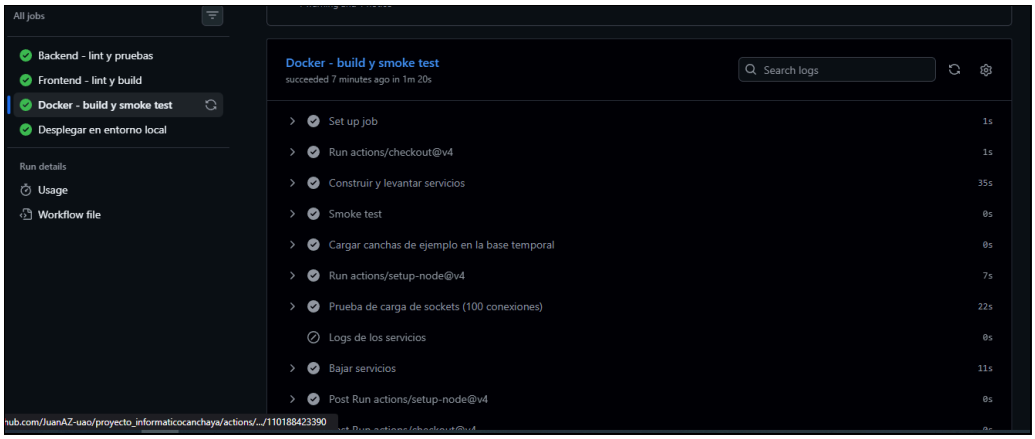


Figura 16. Detalle del job Docker - build y smoke test (incluye la prueba de carga de 100 conexiones).

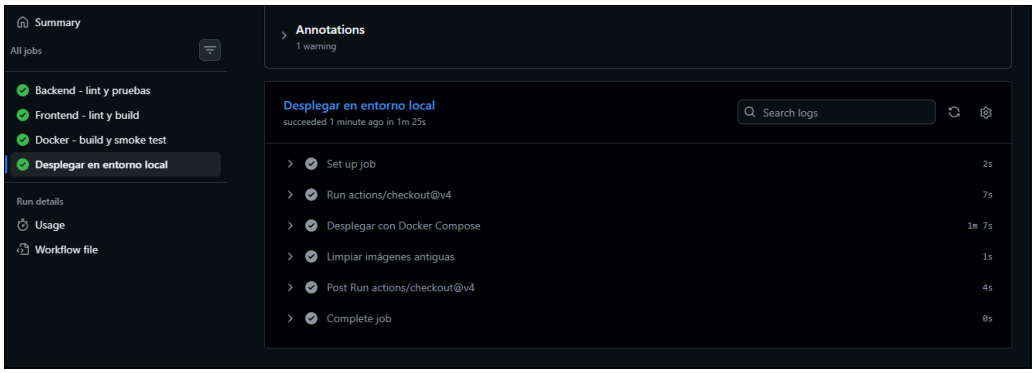


Figura 17. Job Desplegar en entorno local ejecutado en el self-hosted runner.

4.6 Protección de ramas

Estado: no configurada. En el repositorio no está activa la regla «Require a pull request before merging» con «Require approvals» (Settings → Branches → Branch protection rules), de modo que hoy la revisión no está forzada por configuración y el PR #5 se fusionó con checks en rojo. Esta mejora está incluida en la sección 8.

5. PREGUNTAS GUÍA SOBRE LAS PRÁCTICAS DE INGENIERÍA

5.1 ¿Por qué evitar cambios directos en la rama principal del proyecto?

La rama principal debe contener siempre una versión que funciona y que ya fue validada. Un push directo salta la revisión y las pruebas automáticas, impide saber quién cambió qué y por qué, y deja a todo el equipo trabajando sobre código que pudo romperse sin que nadie lo haya revisado. En un equipo de cuatro personas que modifican los mismos módulos (por ejemplo, reservaService.js y su mock), los cambios directos también multiplican los conflictos. Por eso el equipo trabaja cada historia en su propia rama y solo integra en la rama del sprint (sprint_N) por Pull Request; main queda reservada para la versión validada en el Sprint Review. La experiencia del PR #5 lo confirma: un cambio que entró sin revisión suficiente dejó marcadores de conflicto dentro del código y rompió el pipeline hasta corregirlo.

5.2 ¿Cómo funciona nuestro proceso de Pull Request?

1. El integrante crea una rama desde la rama del sprint con el número de su historia (por ejemplo, feature/us-13-historial-reservas) y trabaja con commits pequeños.
2. Actualiza la rama con git fetch y git merge origin/sprint_N, y resuelve los conflictos localmente.
3. Abre el PR hacia sprint_N. El título lleva la historia y la descripción explica qué resuelve y cómo se probó.
4. El pipeline de GitHub Actions corre solo: lint y pruebas del backend con una base de datos real, lint y build del frontend, Quality Gate de SonarQube y construcción de Docker con smoke test.
5. Otro integrante revisa el código, comenta y aprueba o solicita cambios; el autor responde con nuevos commits en la misma rama.
6. Con la aprobación y los checks en verde se fusiona con «Create a merge commit» y se elimina la rama.

Ejemplos reales: el PR #4 (US-13) y el PR #3 (US-11, US-18, US-19 y US-20) siguieron este flujo, con los checks automáticos en verde antes de fusionar (8 de 8). La parte de la revisión por otra persona es la que el equipo aún debe registrar en GitHub.

5.3 ¿Qué papel juega la revisión de código en el control de versiones?

La revisión es el filtro humano entre el trabajo de una persona y la rama compartida. Detecta errores de lógica que las pruebas no cubren, verifica que el cambio cumple los criterios de aceptación de la historia, mantiene un estilo homogéneo y reparte el conocimiento del código entre los integrantes. En nuestro proyecto lo comprobamos con una revisión posterior a la fusión (la regresión de QA): encontró ${numDefectos} defectos en total, entre ellos que la historia US-13 había quedado incompleta tras unir las ramas (el repositorio no tenía la consulta paginada que la pantalla esperaba) y que las pruebas de integración de US-12 fallaban con la validación de horario. Una revisión previa al merge los habría detectado antes de llegar a la rama del sprint. Además, la revisión deja registro en el PR (comentarios, aprobaciones), que es la evidencia de que el cambio no fue aprobado por su propio autor.

5.4 ¿Cómo se fusiona un Pull Request con una rama?

Con el PR abierto y sin conflictos, el revisor pulsa «Merge pull request» (Create a merge commit) en GitHub. GitHub crea un commit de fusión con dos padres (la punta de sprint_N y la punta de la rama de la historia) y sube el resultado a sprint_N. Es equivalente a ejecutar localmente git checkout sprint_4, git merge --no-ff feature/us-13-historial-reservas y git push origin sprint_4. Un ejemplo real es el commit 543243a, «Merge pull request #3 from JuanAZ-uao/US--11,18,19,20», que integró en sprint_4 los commits 548e6ad y ee309ca. Si el PR tiene conflictos, GitHub no permite fusionarlo hasta resolverlos (sección 4.4).

5.5 ¿Qué método de fusión usamos (merge, squash o rebase) y por qué?

Usamos **merge commit** (los PR #3, #4, #5 y #6). Elegimos este método porque: (1) conserva todos los commits de la historia con su autor, de modo que se ve quién hizo qué; (2) deja un punto explícito en el historial donde cada historia entró al sprint, visible con git log --graph, lo que da trazabilidad HU → rama → merge; (3) no reescribe commits, algo importante cuando varias personas comparten ramas, y (4) permite revertir una historia completa con un solo git revert -m 1 sobre el commit de fusión. Descartamos *squash* porque aplasta los commits pequeños en uno solo y perdería el detalle por integrante, y *rebase* porque cambia los hashes de los commits y puede romper el trabajo de quien ya tenga la rama descargada. El PR #2 (US-08) se integró con un fast-forward local y un push, y GitHub lo marcó como Merged; en adelante todos los PR se fusionan desde GitHub con merge commit.

5.6 ¿Cómo resolvimos un conflicto al fusionar ramas y qué haríamos si se presentara otro?

El caso real está documentado en la sección 4.4 (PR #5, historia US-12): se identificaron los archivos con git status y git diff --diff-filter=U, se decidió qué conservar de cada lado, se eliminaron los marcadores, se verificó con las pruebas y se agregaron solo los archivos resueltos antes del commit. Si se presentara otro conflicto, el procedimiento es: (1) no fusionar a ciegas con la opción «Accept both» de la interfaz web sin revisar el resultado; (2) actualizar la rama de la historia con git fetch y git merge origin/sprint_N en local; (3) entender qué intención tiene cada cambio hablando con el autor del otro lado; (4) combinar a mano, buscar marcadores residuales con git grep -n " <<<<<<<" y correr las pruebas; (5) hacer el commit del merge con un mensaje que describa la resolución y subir la rama para que el PR quede sin conflictos. Si no se logra resolver en el sprint, se documenta el motivo y se pide ayuda al autor de la otra rama.

6. CALIDAD DEL INCREMENTO: PRUEBAS, SONARQUBE Y DOCUMENTACIÓN DE LA API

Las prácticas de control de versiones se apoyan en controles de calidad automáticos antes de integrar. Resumen de la regresión completa ejecutada sobre la rama sprint_4 después de unir las historias US-12, US-13 y US-21:

Tabla 8. Resultados de la regresión de QA

Prueba	Cantidad	Resultado
Pruebas unitarias e integración del backend (Jest)	140 pruebas	140 aprobadas · 0 fallidas
Pruebas funcionales (API y sockets)	72 casos	71 aprobados · 0 fallidos · 1 observaciones
Pruebas E2E en navegador	25 casos	25 aprobados · 0 fallidos · 0 observaciones
Pruebas no funcionales	21 casos	16 aprobados · 0 fallidos · 5 observaciones
Smoke test de Docker y pipeline	24 casos	23 aprobados · 0 fallidos · 1 observaciones

SonarQube. Quality Gate aprobado: 0 bugs, 0 vulnerabilidades, 0 puntos críticos de seguridad y 0 code smells; cobertura del backend 51.7 % y duplicación 0.0 %.

Documentación de la API (OpenAPI/Swagger). La especificación está en backend/openapi/openapi.yaml y se publica con Swagger UI en /api/docs. Documenta los 14 endpoints implementados (cobertura del 100 %, con el 90 % como mínimo) con ejemplos de respuesta, los eventos de Socket.IO y el botón Authorize con token JWT. Se valida con Spectral y con una prueba de contrato que ejecuta los ejemplos contra el backend real; ambas comprobaciones forman parte del pipeline.

Los resultados completos están en los documentos de QA de la carpeta docs/testing del repositorio (plan, funcionales, no funcionales, E2E, smoke de Docker, informe de defectos, SonarQube y verificación de la documentación de la API).

7. DECLARACIÓN DE USO DE INTELIGENCIA ARTIFICIAL

El equipo declara que utilizó una herramienta de inteligencia artificial generativa, **Claude Code** (asistente de programación de Anthropic, integrado en Visual Studio Code), como apoyo en el desarrollo del proyecto y en la elaboración de este entregable.

7.1 Para qué se usó

- Implementar y corregir historias de usuario del backend y del frontend (US-10, US-11, US-17, US-18, US-19 y US-20) y completar la historia US-13 después de la fusión de ramas.
- Resolver los conflictos de fusión y los errores que aparecieron al unir las ramas, y volver a ejecutar las pruebas.
- Diseñar, programar y ejecutar el plan de pruebas de QA (funcionales, no funcionales, E2E y smoke de Docker), y generar los documentos de resultados.
- Configurar el pipeline de CI/CD, la contenerización con Docker y el análisis con SonarQube.
- Redactar la documentación técnica de la API (OpenAPI/Swagger), el README y este informe a partir de datos reales del repositorio y de la API pública de GitHub.

7.2 Razones que soportan su uso

- Reducir el tiempo de tareas repetitivas y de configuración (pipeline, Docker, documentación) para dedicar más tiempo a las historias de usuario.
- Ampliar la cobertura de pruebas: la IA permitió ejecutar cientos de verificaciones que habrían tomado mucho más tiempo a mano y encontró defectos reales que el equipo corrigió.
- Mantener documentación coherente con el código y con los criterios de aceptación.
- Aprender buenas prácticas (Conventional Commits, flujo de Pull Request, OpenAPI) viendo su aplicación sobre el propio proyecto.

7.3 Controles aplicados

- Las decisiones de alcance, prioridad y arquitectura las tomó el equipo; la IA ejecutó solo lo solicitado.

- Todo código generado se ejecutó con pruebas automáticas antes de integrarse, y los resultados se registraron en los documentos de QA.
- Los datos de este informe (PR, commits, autores, fechas y estado de las revisiones) se tomaron del historial de Git y de la API de GitHub; no se inventaron revisiones ni aprobaciones, y lo que falta se declara como pendiente.
- Las credenciales reales (archivo .env) nunca se compartieron ni se versionaron.
- El equipo es responsable de revisar, entender y defender lo entregado; la IA no sustituye la revisión por pares ni la aprobación de los Pull Requests.

8. ESTADO DE CUMPLIMIENTO DE LA GUÍA Y MEJORAS

Tabla 9. Cumplimiento de los criterios del entregable

Criterio	Estado	Evidencia o acción pendiente
Captura del tablero de la Sprint Planning con las HU de los sprints 3 y 4	Cumplido	Figuras 1 y 2 y tabla 1.
README con descripción, stack, estrategia de ramas (diagrama) y convención de commits	Cumplido	Sección 2.1 y README.md del repositorio.
.gitignore adecuado al stack	Cumplido	Sección 2.2.
Tabla resumen de PR con #PR, HU, autor, revisor, método de merge y URL	Cumplido con observación	Tabla 3. El campo «Revisor» indica que no hay revisión registrada en GitHub.
Commits pequeños, semánticos y trazables a su HU	Parcial	Trazabilidad por rama y PR (sección 4.1). Los commits agrupan varias HU y varios mensajes no siguen la convención; la convención queda definida y se aplica desde ahora.
Ramas de feature vinculadas a su HU y al menos un PR fusionado por integrante	Cumplido	Sección 4.2: PR #2 y #4 (Castillo), #3 (Panesso), #5 (Ríos), #6 (Zapata). Los PR #5 y #6 los abrió la cuenta JuanAZ-uao.
Al menos un PR con comentarios o cambios solicitados antes de aprobarse	Pendiente	Sin evidencia en GitHub. Acción: ver el procedimiento de abajo.
Simulación de un conflicto y su corrección	Cumplido	Sección 4.4 (PR #5).
Declaración de uso de IA	Cumplido	Sección 7.

Criterio	Estado	Evidencia o acción pendiente
Branch protection activa (evidencia recomendada por la guía)	Pendiente	No configurada. Acción: ver el procedimiento de abajo.

Acciones del equipo antes de entregar

1. **PR con revisión real.** Abrir el PR de la siguiente historia desde su rama hacia sprint_4; otro integrante lo revisa en GitHub, deja al menos un comentario general y uno sobre una línea específica en «Files changed», pulsa «Request changes», el autor corrige con un nuevo commit y el revisor pulsa «Approve». Después se fusiona con merge commit y se captura la conversación y el panel Reviewers.
2. **Branch protection.** En GitHub: Settings → Branches → Add branch protection rule sobre sprint_4 (y main cuando exista), con «Require a pull request before merging» y «Require approvals» (1) marcados, más «Require status checks to pass». Capturar la pantalla.
3. **Commits atómicos.** Aplicar desde ahora la convención tipo(alcance): descripción, un cambio lógico por commit.

Cuando existan esas evidencias, se guardan como pr-con-revision.png (y opcionalmente pr-con-revision-lineas.png) en la carpeta docs/informe/evidencias y se regenera este informe con node docs/informe/generar-informe.mjs.